

MLOps Production Architecture Design

- Dataset storage

Instead of using a local hard drive for storing the images of pedestrians or data. I would use the **AWS S3** service. It is a cloud folder to store data. It is a highly secure, scalable cloud storage system. This service will organise the data into folders. Here, I will have folders for training and testing. I will also turn on S3 versioning so that if any image or data is added or modified, then I can track what the data looked like at any point in time. This will help other engineers to pull the exact same dataset to duplicate a training run.

- Experiment tracking

For experiment tracking, I can use **AWS EC2**. To ensure that the progress is shared with the team, I will run the MLflow Tracking Server on a central **AWS EC2** cloud computer instead of keeping it local. This cloud server can connect to a secure database to connect to or log into the training configurations (The cloud server will save the hyperparameter configurations). Configurations such as learning rate, epochs, and batch size. The cloud server can also connect to metrics such as loss or mAP. It would allow any team member to view, search, and compare every training run through a single dashboard.

- Model Artifact Management

For Model Artifact Management, I will use **AWS S3**. After the training gets completed, the weight file (Final model weight file (final_model.pt)) will be updated automatically and saved into a protected S3 bucket. It will help to version and track the model from experiment to production. The bucket will also get connected to the MLflow Model Registry. Here, the model will be versioned and tagged with its current status (Such as Staging for testing or production for live deployment)

- Automated training workflows

Here, I will automate the training process completely using a CI/CD pipeline. When the changes in the code are pushed to the repository, a script will automatically build myDockerfile and save it inside Amazon **ECR** (ECR is a cloud locker for docker images). After this, I will use Amazon SageMaker. Amazon SageMaker pipelines will automatically use a powerful GPU, docker image from ECR, and use the data from S3. Then, it will run the entire training pipeline, and immediately shut down the machine when finished (It will never pay for idle cloud costs).

- Monitoring and retraining strategy

Here, once the model is live, I will use the **AWS CloudWatch** to monitor its real-world performance. It also checks if the new data or new images start looking different (different such as shifts in lighting or angle that might cause the model to miss pedestrians). If CloudWatch notices that the model's accuracy or confidence scores drop below a certain threshold, then automatically an alarm will be triggered. This alarm will start a new or fresh SageMaker training session using the new images or data to update the model.